Concept:

This project optimizes customer retention for a subscription-based service by predicting customer churn. It analyzes key indicators such as customer behavior, engagement, and spending patterns, using machine learning models to identify at-risk customers.

Tools Used:

- **Python** for data analysis and machine learning.
- **Pandas**, **NumPy** for data manipulation.
- **Matplotlib**, **Seaborn** for visualization.
- **Scikit-learn** for predictive modeling.

Implementation:

The solution includes generating synthetic data, performing exploratory data analysis (EDA), building a Random Forest model, and visualizing churn-related insights.

In [1]:
```
#Step 1: Data Generation (scripts/generate_data.py), not shown here
#---------------shown in this cell---------------
#Step 2: Exploratory Data Analysis (EDA) (notebooks/customer_retention_analy
#Cell 1: Import Libraries

# Importing necessary libraries for analysis
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# To display plots inline
%matplotlib inline
```

In [2]:
```
#Cell 2: Load Data
# Load the synthetic customer data
data = pd.read_csv('../data/customer_data.csv')

# Show the first few rows of the dataset
data.head()
```

Out[2]:

	customer_id	tenure	avg_monthly_spend	last_login_days	engagement_sco
0	1	7	52.621952	8	2.51419
1	2	20	31.969328	27	69.44504
2	3	15	153.401717	29	8.22070
3	4	11	123.405160	13	91.06765
4	5	8	171.529180	18	46.44332

In [3]:
```
#Cell 3: Basic Data Overview
# Data overview
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype  
---  ------             --------------  -----  
 0   customer_id        1000 non-null   int64  
 1   tenure             1000 non-null   int64  
 2   avg_monthly_spend  1000 non-null   float64
 3   last_login_days    1000 non-null   int64  
 4   engagement_score   1000 non-null   float64
 5   is_churn           1000 non-null   int64  
dtypes: float64(2), int64(4)
memory usage: 47.0 KB
```

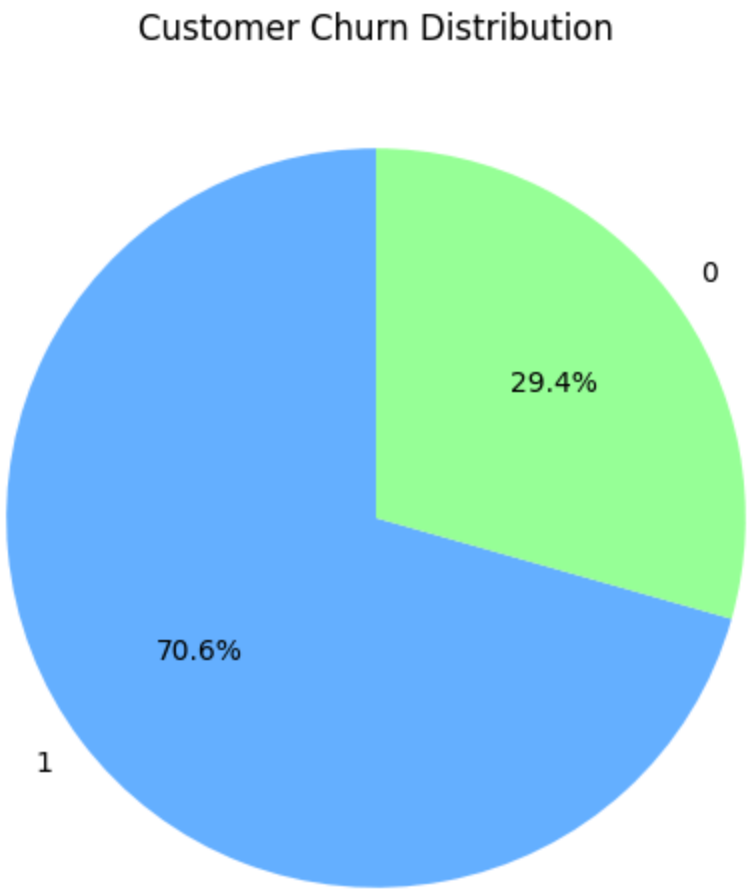
In [4]:
```
#Cell 4: Descriptive Statistics
# Summary statistics
data.describe()
```

Out[4]:

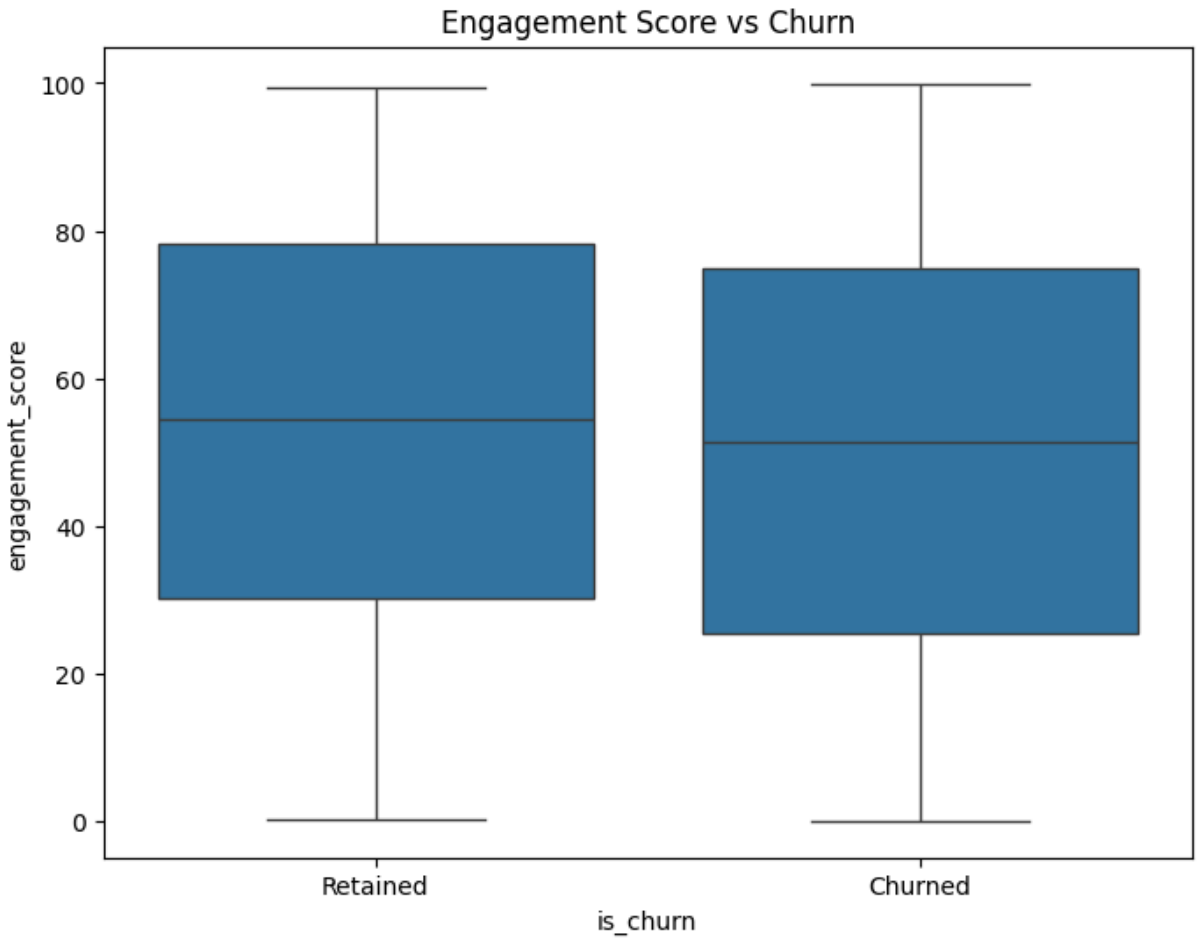
	customer_id	tenure	avg_monthly_spend	last_login_days	engage
count	1000.000000	1000.000000	1000.000000	1000.000000	1
mean	500.500000	11.845000	110.396157	14.872000	
std	288.819436	6.854046	52.525338	8.687064	
min	1.000000	1.000000	20.833764	0.000000	
25%	250.750000	6.000000	62.425706	7.000000	
50%	500.500000	12.000000	112.475528	15.000000	
75%	750.250000	18.000000	155.116912	22.000000	
max	1000.000000	23.000000	199.894471	29.000000	

In [5]:
```
#Cell 5: Visualizing Churn Distribution (Pie Chart)
# Visualize churn distribution (0 = retained, 1 = churned)
plt.figure(figsize=(8, 6))
data['is_churn'].value_counts().plot(kind='pie', autopct='%1.1f%%', startang
plt.title('Customer Churn Distribution')
```

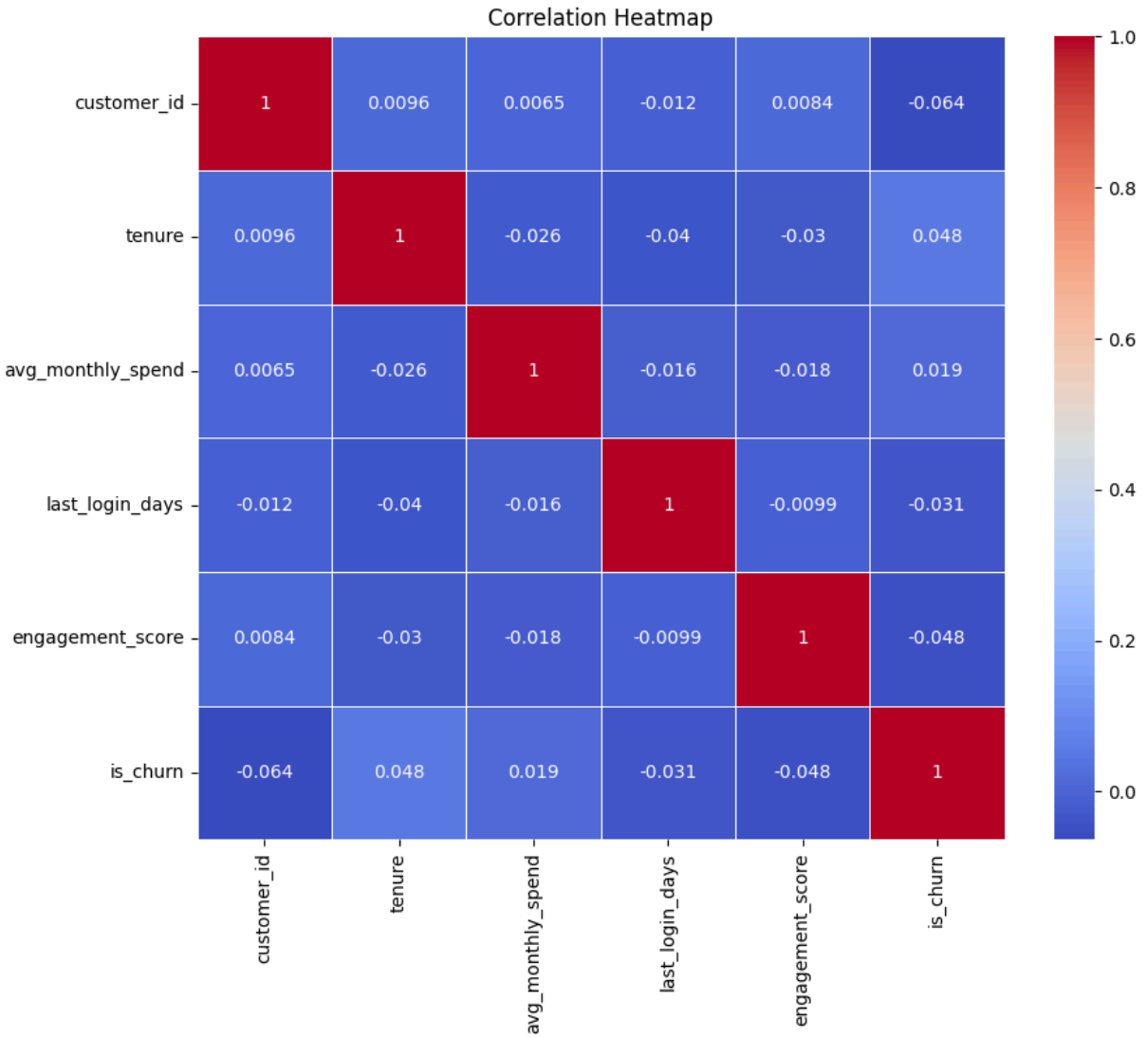
```
plt.ylabel('')
plt.show()
```



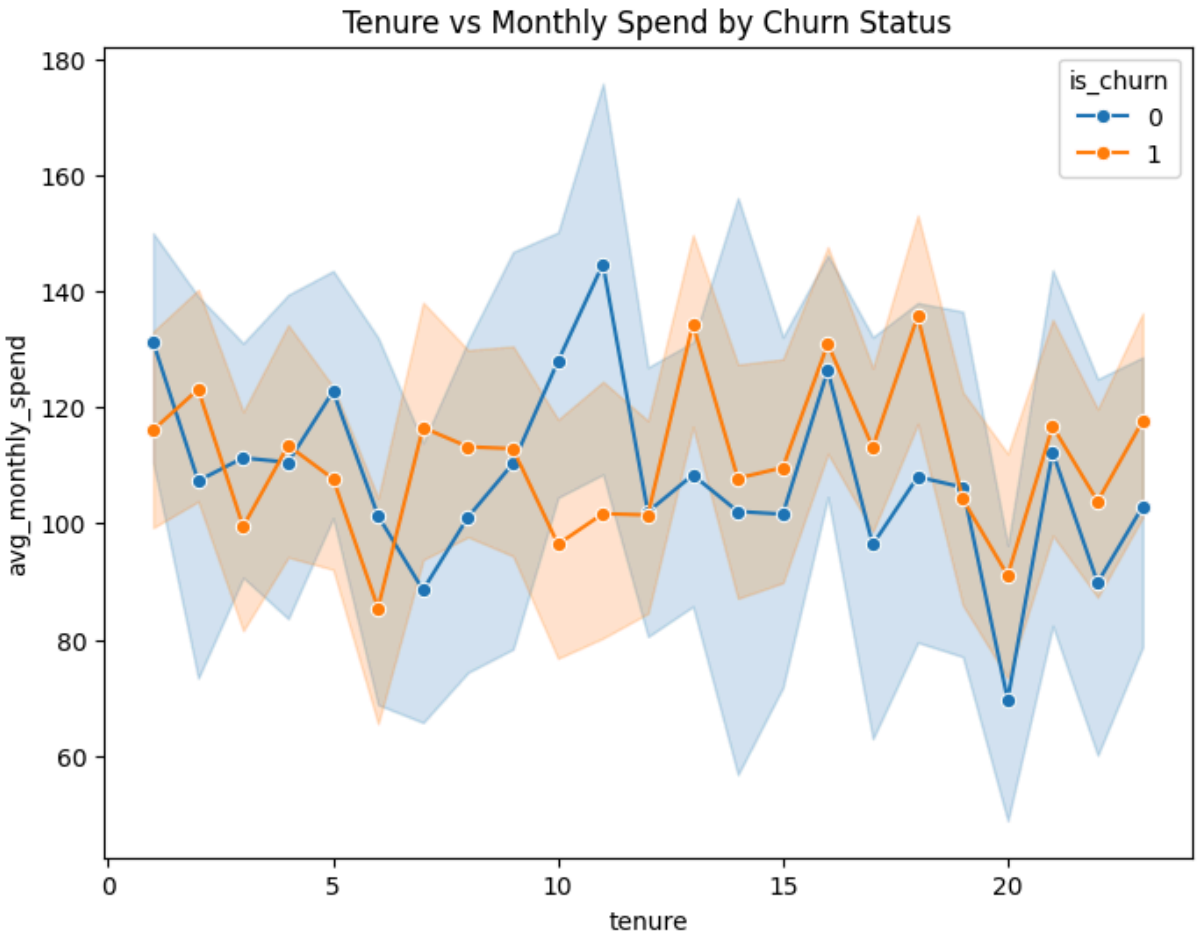
In [6]:
```
#Cell 6: Engagement vs Churn (Bar Chart)
# Compare engagement score between churned and retained customers
plt.figure(figsize=(8, 6))
sns.boxplot(x='is_churn', y='engagement_score', data=data)
plt.title('Engagement Score vs Churn')
plt.xticks([0, 1], ['Retained', 'Churned'])
plt.show()
```



In [7]:
```
#Cell 7: Correlation Heatmap
# Visualize correlation heatmap
plt.figure(figsize=(10, 8))
sns.heatmap(data.corr(), annot=True, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Heatmap')
plt.show()
```



In [8]:
```
#Cell 8: Tenure vs Churn (Line Plot)
# Average tenure by churn status
plt.figure(figsize=(8, 6))
sns.lineplot(x='tenure', y='avg_monthly_spend', hue='is_churn', data=data, m
plt.title('Tenure vs Monthly Spend by Churn Status')
plt.show()
```



In [9]:
```
#Step 3: Data Preprocessing (Feature Engineering and Encoding)

# Convert categorical data into numerical
# For simplicity, let's encode 'is_churn' into binary values where 0 = retai
from sklearn.model_selection import train_test_split

# Split data into features (X) and target (y)
X = data[['tenure', 'avg_monthly_spend', 'last_login_days', 'engagement_scor
y = data['is_churn']

# Split the data into train and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, rar

# Display shapes of resulting splits
(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

Out[9]:
```
((800, 4), (200, 4), (800,), (200,))
```

In [10]:
```
#Step 4: Build Predictive Model (Machine Learning)

#Cell 1: Import Model Libraries
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusior
from sklearn.preprocessing import StandardScaler
```

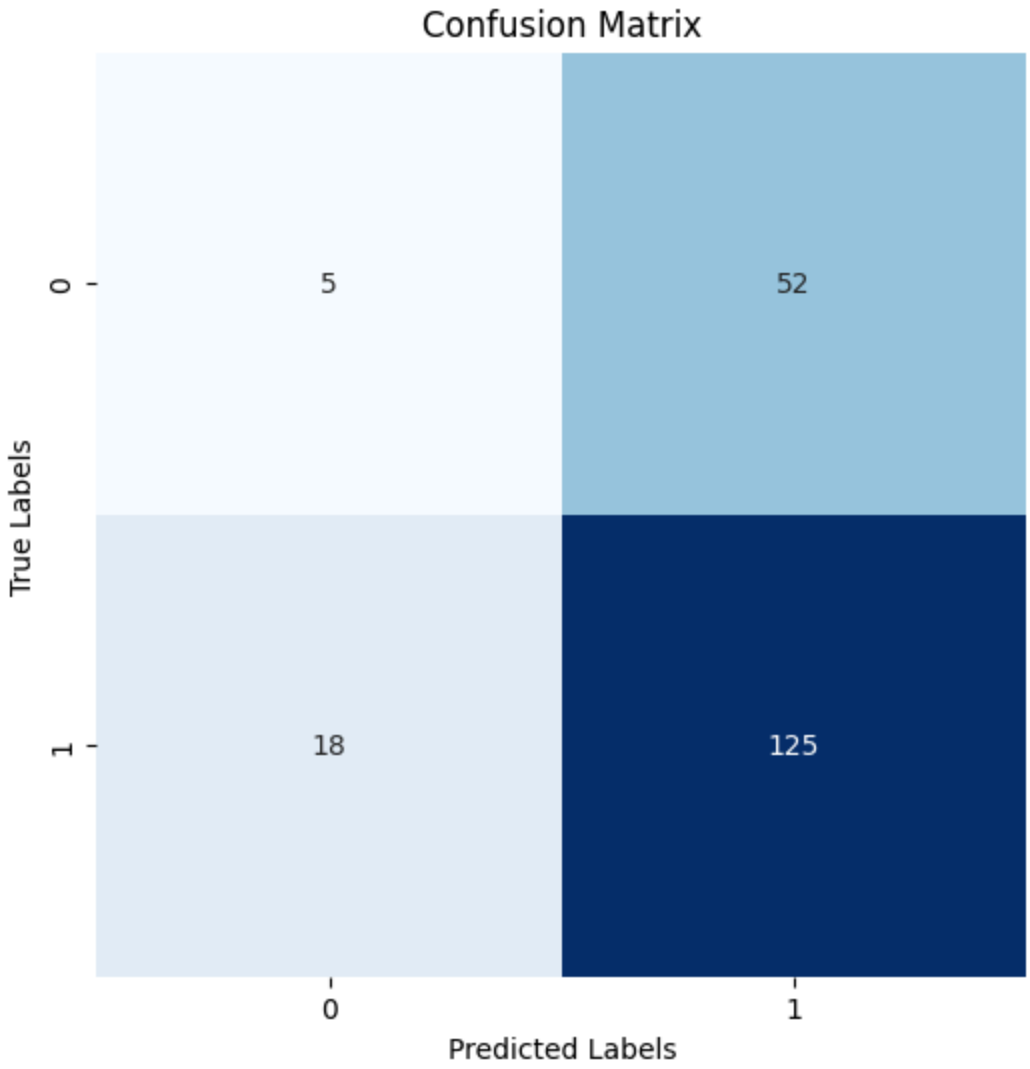
```
# Initialize a RandomForestClassifier
model = RandomForestClassifier(n_estimators=100, random_state=42)
```

In [11]:
```
#Cell 2: Data Scaling
# Normalize the data using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

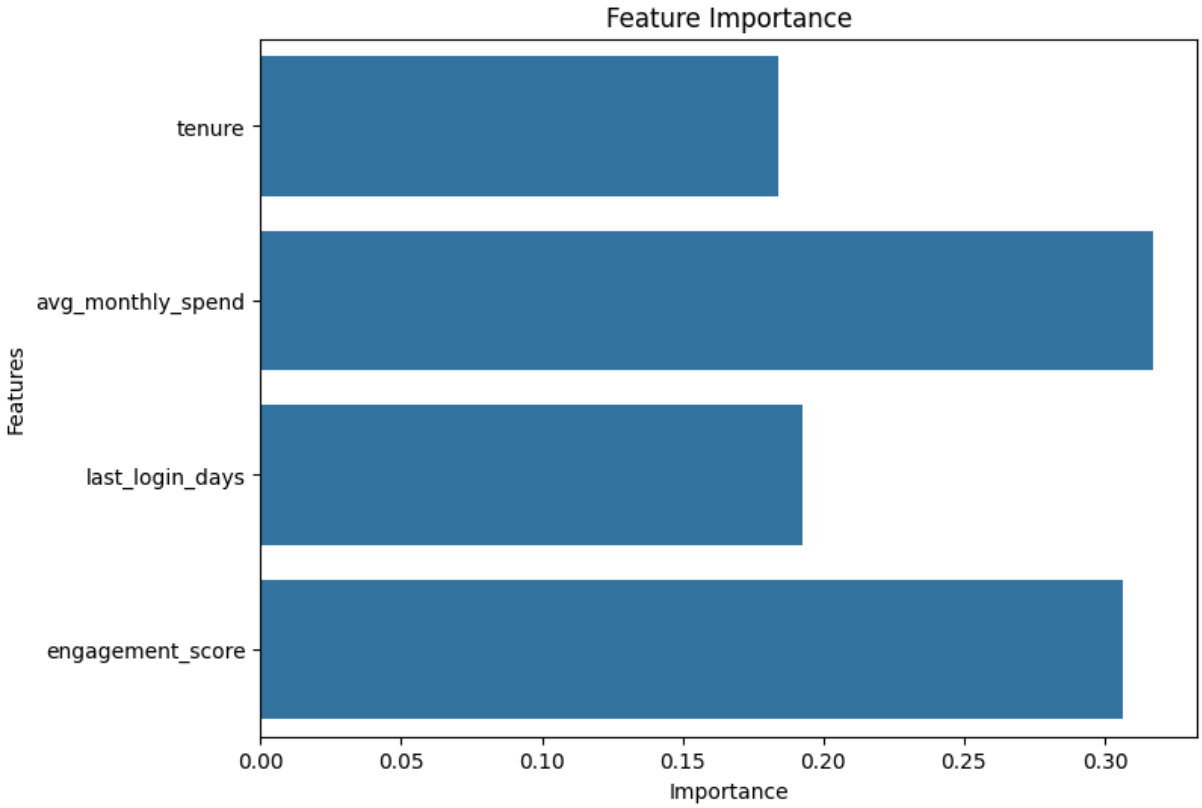
In [12]:
```
#Cell 3: Model Training
# Train the model
model.fit(X_train_scaled, y_train)

# Predict on test data
y_pred = model.predict(X_test_scaled)
```

In [13]:
```
#Cell 4: Evaluate Model
# Evaluate model performance
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy * 100:.2f}%')

# Detailed classification report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False)
plt.title('Confusion Matrix')
plt.ylabel('True Labels')
plt.xlabel('Predicted Labels')
plt.show()
```

```
Accuracy: 65.00%

Classification Report:
              precision    recall  f1-score   support

           0       0.22      0.09      0.12        57
           1       0.71      0.87      0.78       143

    accuracy                           0.65       200
   macro avg       0.46      0.48      0.45       200
weighted avg       0.57      0.65      0.59       200
```



In [14]:
```python
# Step 4: Model Building (after training the model)
# Model has been trained, and we can extract the feature importances now

# Extract feature importances from the trained model
feature_importances = model.feature_importances_

# Get feature names
features = X.columns

# Step 5: Plot Feature Importance
plt.figure(figsize=(8, 6))
sns.barplot(x=feature_importances, y=features)
plt.title('Feature Importance')
plt.xlabel('Importance')
plt.ylabel('Features')
plt.show()
```



In [15]:
```
#Cell 2: Predict At-Risk Customers
# Predict the probability of churn for each customer in the test set
y_prob = model.predict_proba(X_test_scaled)[:, 1]  # probability of churn (c

# Add probabilities to the DataFrame for analysis
predicted_churn = pd.DataFrame({
    'customer_id': X_test.index + 1,  # Customer IDs are 1-based
    'predicted_churn_prob': y_prob
})

# View top customers with highest risk of churn
predicted_churn.sort_values(by='predicted_churn_prob', ascending=False).head
```

Out[15]:

	customer_id	predicted_churn_prob
87	715	0.99
93	219	0.94
50	571	0.92
58	222	0.92
184	846	0.92
153	430	0.91
166	299	0.90
80	680	0.90
103	257	0.89
174	487	0.89

In [16]:
```
#Step 6: Saving and Deploying the Model

#Cell 1: Save the Model
import joblib

# Save the trained model for future use
joblib.dump(model, 'customer_retention_model.pkl')

# Save the scaler
joblib.dump(scaler, 'scaler.pkl')

print("Model and scaler saved!")
```

```
Model and scaler saved!
```

In [17]:
```
#The End
```